Visualizing CNN Feature Maps - Understanding Deep Learning Representations

Name: Yaswanth

Introduction

Convolutional Neural Networks (CNNs) have become a cornerstone of modern deep learning, particularly in image recognition and computer vision tasks. Their ability to automatically learn hierarchical representations from raw pixel data has significantly outperformed traditional machine learning approaches. However, despite their high performance, CNNs are often considered “black box” models because their internal decision-making processes are not easily interpretable.

Understanding how CNNs learn and process information is essential for improving model design, debugging errors, and building trust in real-world applications. One effective way to achieve this is through the visualization of feature maps, which reveal how different layers respond to input images.

This tutorial focuses on visualizing CNN feature maps to gain insight into how representations evolve across layers. The experiment is conducted using the MNIST dataset, which consists of grayscale images of handwritten digits. Due to its simplicity and well-defined patterns, MNIST is ideal for exploring internal CNN behaviour.

Understanding CNN Feature Learning

CNNs learn features in a hierarchical manner, with each layer capturing increasingly complex patterns.

Early Layers: Low-Level Features

The first convolutional layers detect basic visual elements such as:

- Edges
- Corners
- Simple textures

These features are general and appear in all images.

Intermediate Layers: Pattern Formation

As we move deeper into the network, layers begin to combine simple features into more meaningful patterns:

- Curves
- Shapes
- Digit components

These representations are more specific to the dataset.

Deep Layers: Abstract Representations

The deepest convolutional layers capture high-level abstractions:

- Complete digit structures
- Class-specific features

These representations are highly specialized and directly influence classification decisions.

3. Experimental Setup

Dataset:

The MNIST dataset contains:

- 70,000 grayscale images
- Resolution: 28×28 pixels
- 10 classes (digits 0–9)

All pixel values were normalized to the range [0,1] to improve training efficiency.

Model Architecture:

A straightforward CNN was built utilizing three convolutional layers:

- The first convolutional layer uses 32 filters that are used to identify edges.
- Conv Layer 2 helps in learning patterns using 64 filters.
- Conv Layer 3 contains 128 filters (abstract features).

To shrink the spatial dimensions, max pooling layers were utilized, followed by layered fully connected for classification.

Training Configuration:

The model was trained using:

- **Optimizer**: Adam
- **Loss**: Sparse categorical cross-entropy
- **Epochs**: 3 (for efficiency)
- **Batch size**: 128.

The goal was not maximum accuracy but meaningful feature extraction.

4. Feature Map Extraction

To analyse internal representations, a secondary model was created to output intermediate layer activations. This allows us to inspect how each convolutional layer transforms the input image.

A sample test image was passed through the network, and feature maps were extracted from all convolutional layers.

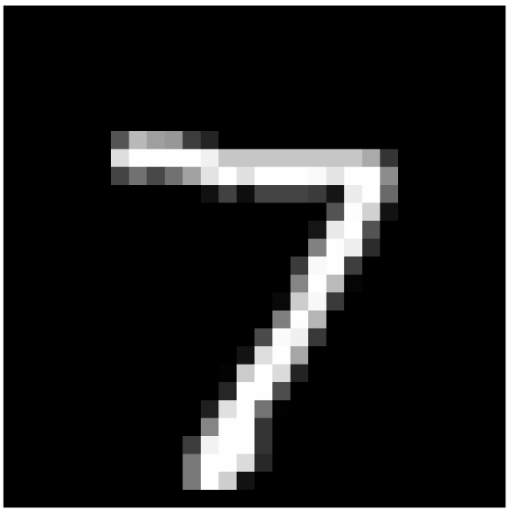
5. Results and Visualization

Input Image:

The first figure shows the original handwritten digit. This serves as the reference input for all subsequent visualizations.

The simplicity of the MNIST images allows us to clearly observe how features are progressively extracted.

Input Image



Feature Maps: First Layer

The maps emerge early in the model and help identify patterns of the edge and intensity.

- Areas with strong filter activation lights up in bright colours.
- Low activation is represented by dark regions.

These filters are sensitive to simple, basic structures like vertical and horizontal strokes that make up digits.

Feature Maps - Layer 1

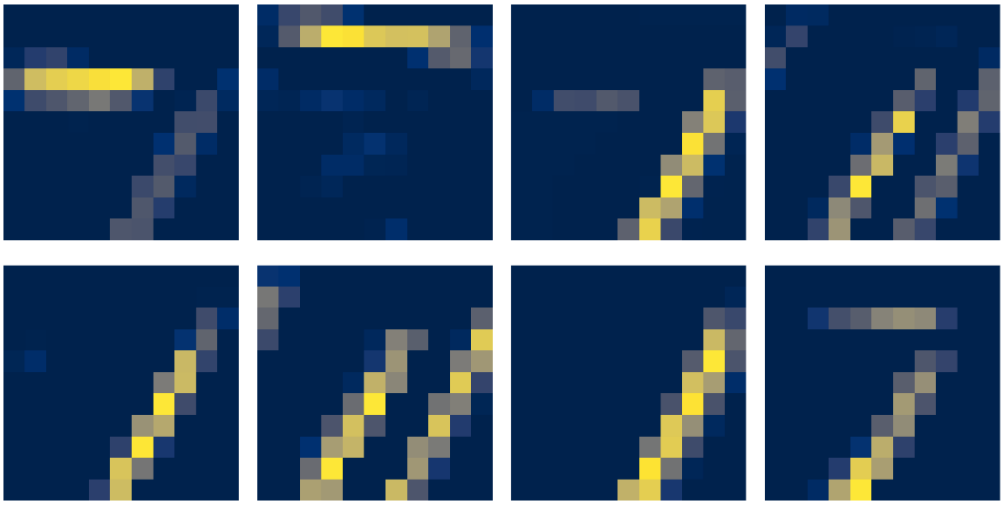
Feature Maps: Second Layer

The second layer combines basic features into more complex patterns.

- Some filters detect curves or loops.
- Others focus on specific digit segments.

Compared to the first layer, these feature maps are more structured and localized.

Feature Maps - Layer 2



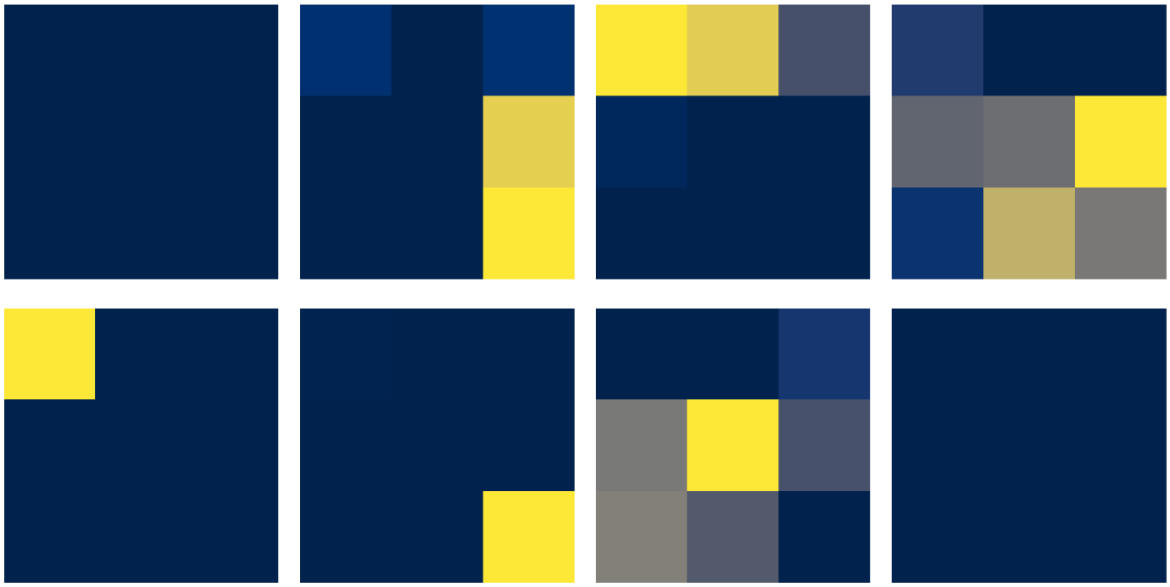
Feature Maps: Deep Layer

The deepest convolutional layer produces highly abstract representations.

- Feature maps become less interpretable visually.
- Activations focus on specific digit characteristics.

These representations are tailored for classification and capture the overall structure of the digit rather than individual components.

Feature Maps - Deep Layer



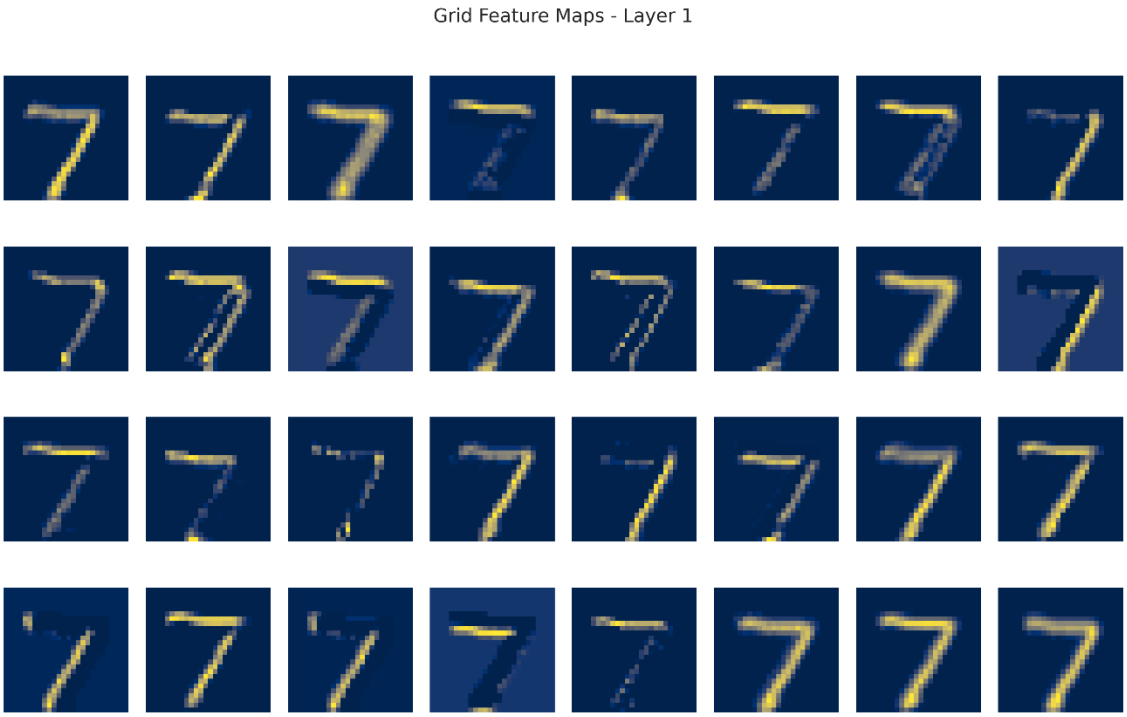
Grid Visualization of Feature Maps:

The grid visualization provides a comprehensive overview of all filters within a layer.

- Each subplot represents one filter.
- The grid structure makes it easier to compare patterns.

In early layers, the grid shows a variety of edge detectors.
In deeper layers, the grid reveals sparse and highly focused activations.

This visualization clearly demonstrates the hierarchical nature of CNN learning, where representations evolve from general to specific.



Discussion

The findings show how CNNs learn progressively. The initial layers' feature maps have captured generic features like edges and a few gradients. The properties are not unique to any digit; they are building blocks for further layers.

With increased depth, feature maps become more specialized. Intermediate layers start recognizing meaningful features like curves and parts of the digits. The model is combining features from different layers which allows them to learn.

The lowest layers have accretions that are very abstract and closely related to classification. At this stage of digit recognition, the network analyses shapes instead of individual pixels or edges; it detects similar features and examines the perceived digit.

It is noted that deeper feature maps become harder to visually interpret. It shows the balance between interpretability and performance in deep learning models. The lower layers are easy to understand, while the deeper layers encode complex relationships.

Grid visualization is important to improve interpretability. Displaying several filters at once helps us see patterns and variations in the feature maps. Such information allows for a more complete perspective of network operation.

We also learn that CNNs automatically learn hierarchical representations, thus negating manual feature engineering. Due to this ability, they become extremely powerful. But it also highlights the significance of some interpretability tool like feature map visualization.

Nonetheless, this approach has limitations as well. The MNIST dataset is simple, and it is believed that the patterns learned may not work on more complex datasets. Moreover, visualizing feature maps in a neural network does not refer to the specific decision taken by the model.

Limitations

- The MNIST is simple and may not represent real-world image complexity.
- The CNN architecture used is shallow compared to modern models like ResNet or EfficientNet.
- Feature map visualization provides only partial interpretability and does not fully explain model decisions.
- Only a single input image was analysed, limiting the generality of observations.
- Short training duration may result in less refined feature representations.

Conclusion

- CNNs learn from simple edges to complex patterns features.
- Feature map visualization gives insights into internal representations.
- The earlier layers of CNN exhibit high-level features like edges and textures.
- We can understand all filters at once because of Grid visualization.
- Understanding feature maps helps improve model design and debugging.
- There will be better performance with this approach and interpretability.

Future Work

- Apply visualization techniques to more complex datasets such as CIFAR-10
- Use advanced interpretability methods such as Grad-CAM
- Explore deeper architectures like ResNet.
- Analyse feature maps across different classes
- Combine feature visualization with model explainability techniques.

References:

LeCun, Y., Bottou, L., Bengio, Y. and Haffner, P., 1998. Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), pp.2278–2324.

Goodfellow, I., Bengio, Y. and Courville, A., 2016. Deep Learning. Cambridge, MA: MIT Press.

Chollet, F., 2017. Deep Learning with Python. New York: Manning Publications.

Zeiler, M.D. and Fergus, R., 2014. Visualizing and understanding convolutional networks. In: European Conference on Computer Vision (ECCV), pp.818–833.

LeCun, Y., Cortes, C. and Burges, C.J.C., n.d. MNIST handwritten digit database. Available at: http://yann.lecun.com/exdb/mnist/